Distributed Multi-Model Analytics for E-commerce Data

Student: re
Course: Big Data Analytics
Date: 1 November 2026

1. Architecture and business objective

This project analyses a synthetic 90-day e-commerce data set of users, products, sessions, and transactions. The objective is to identify revenue-driving product categories, product affinities, customer cohorts, and acquisition channels that convert browsing to purchases. The architecture separates operational/document workloads from high-volume event retrieval and uses Spark for joins and batch analytics.

```
JSON generator -> MongoDB: users, products, transactions
              -> HBase: flattened page-view events, product-day metrics
MongoDB + HBase -> Spark batch / Spark SQL -> CSV results -> static charts
```

MongoDB holds complete business documents: embedded transaction line items are read together with purchase metadata, while product and user documents support catalogue/profile lookups. HBase holds sparse, append-oriented browsing events. Spark processes all JSON partitions and logically combines the two systems.

2. Data modeling and technology selection

MongoDB

Four collections are used. `categories` stores each category with its embedded subcategory list; `products` has `_id = product_id`, name, `category_id`, inventory fields, and the embedded `price_history` array. The optional product subcategory field remains nullable because the supplied generator omits it. `users` has `_id = user_id`, nested geographic data, registration timestamp, and last-active timestamp. `transactions` has `_id = transaction_id`, user/session IDs, order totals/status, and an embedded `items` array. Embedding items avoids a join for order detail, which is normally read as a single business object.

Indexes are `products(category_id, is_active)`, `users(geo_data.country)`, `transactions(timestamp desc)`, `transactions(user_id, timestamp desc)`, and a multikey `transactions(items.product_id)`. The first aggregation unwinds items, joins product category, and calculates category revenue and units. The second implements recency/frequency/monetary segmentation from transactions. The exact code is in `mongodb/aggregations.py`.

MongoDB execution status: MongoDB aggregation results are pending. The local Docker engine was unavailable during this run, so no MongoDB counts or segment results are claimed here. Run the commands in `README.md` after starting MongoDB and replace this note with the aggregation output.

HBase

`user_events` has row key `user_id|ISO-8601 timestamp|session_id|sequence` and two column families: `event` (type, session ID, optional product ID) and `context` (referrer and device). A prefix range scan therefore retrieves one user's chronological activity without scanning other users. Columns are sparse: non-product pages simply have no product qualifier.

`product_daily_metrics` has key `product_id|date` and `metrics:views` (with additional counters such as cart additions/purchases readily added). This table supports efficient one-product time-range scans. The design duplicates a small derived metric intentionally: HBase is query-first and does not support joins. The schema and loader are `hbase/schema.hbase` and `hbase/load_sessions.py`.

MongoDB is superior for nested catalogue and transaction documents plus flexible aggregation. HBase is superior for many sparse events and predictable time-range access. The duplication is a deliberate trade-off for serving latency.

3. Spark batch processing and SQL

`spark/analytics.py` reads all JSON partitions, parses timestamps, filters invalid transaction rows, and explodes each embedded item into a normalized line item table. This makes revenue, basket pairs, and cohort calculations distributable. Revenue uses the embedded line-item subtotals before any order-level discount, because the source does not allocate discounts to individual products. Recommendation candidates are pairs of distinct products from the same order; canonical ordering (`left < right`) prevents double counting.

The cohort analysis joins purchase lines to users, groups customers by registration month, and sums revenue by later purchase month. Spark SQL implements the funnel query: session records represent HBase browsing data and transaction records represent MongoDB purchases. It groups by referrer and calculates session, engaged-session, converted-session, linked-purchase counts, and conversion rate.

For scale, the implementation reads session files from an explicit file list and selects only required columns before joins. It streams compact aggregate tables to CSV from the driver to avoid Hadoop's native Windows file-output dependency; on a cluster, use Spark's distributed CSV writer instead. Production runs should partition event data by date, broadcast the small product catalogue when appropriate, persist reused normalized tables, and avoid collecting large frames on the driver.

4. Integrated analysis: acquisition-to-purchase funnel

Business question: Which acquisition channel produces the greatest share of converting sessions, and how should marketing budget be prioritized?

HBase supplies page-view/session engagement and referrer data. MongoDB supplies the linked purchase records. Spark joins `session_id`, aggregates by referrer, and produces a conversion percentage. This cannot be efficiently answered by performing a cross-system nested loop; Spark moves the computation to a parallel join and returns a compact result table. The `funnel_by_referrer` output and Figure 3 are the evidence for the decision.

Measured finding (10,000-session sample): email had the highest conversion rate at 10.76% (219 of 2,036 sessions). Search engine traffic had the lowest rate at 9.15% (182 of 1,990 sessions). Email also had the most sessions in this sample, so it led on both volume and conversion rate. The rates are close (a 1.61 percentage point gap), so the sample supports monitoring email performance rather than making a major budget shift without cost and significance data.

5. Visualisations and findings

The following figures were generated from the Spark outputs for the 10,000- session sample. They show descriptive results from synthetic data.

- `cat_019` leads category revenue at 126,677.59 across 559 units, followed by `cat_013` at 112,609.70 and `cat_007` at 103,383.65.
- Product-pair frequency gives candidates for frequently bought together. The top pair appears in three orders, so evidence is sparse at this sample size.
- Email leads channel conversion at 10.76%; search engine traffic is lowest at 9.15%. Channel counts are around 2,000 sessions each.
- Cohort revenue spans 27 cohort/purchase-month combinations. The short synthetic activity window limits conclusions about long-term retention.

Figure 1. Category revenue

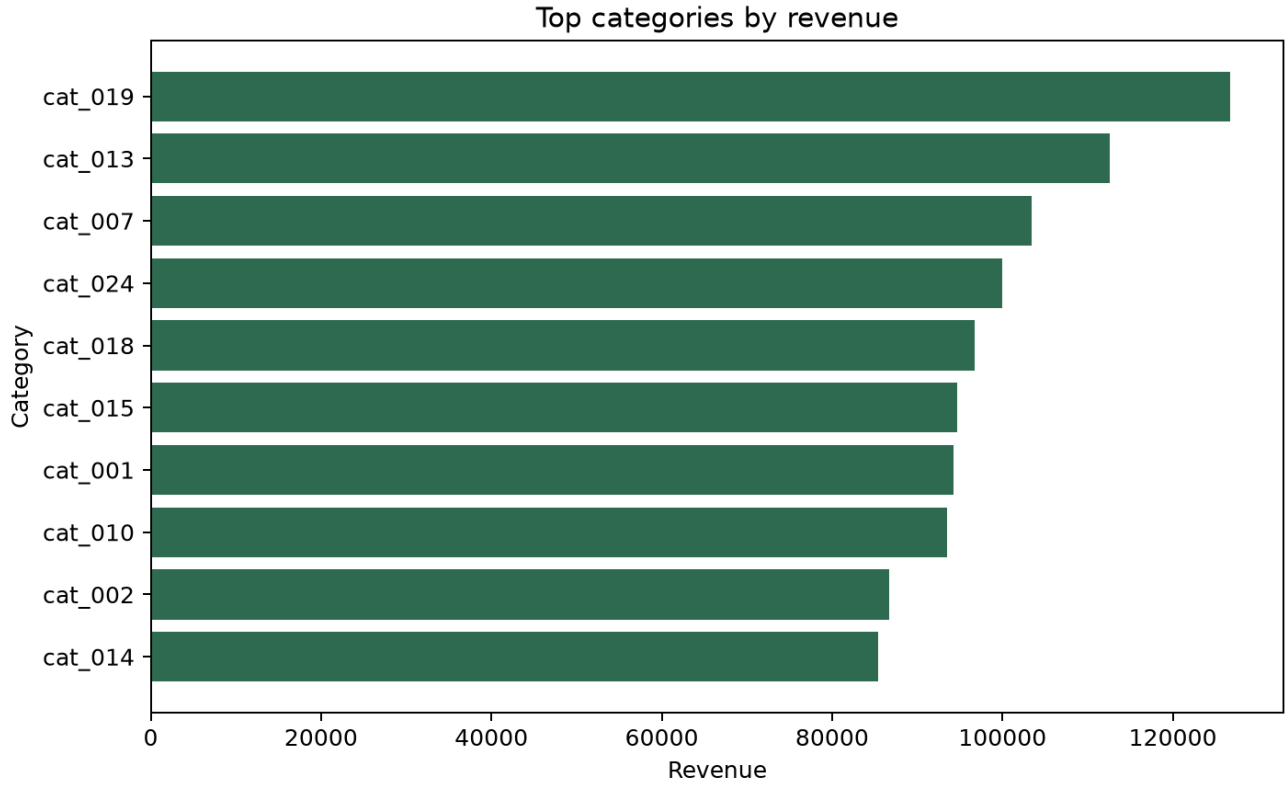


Figure 2. Frequently purchased product pairs

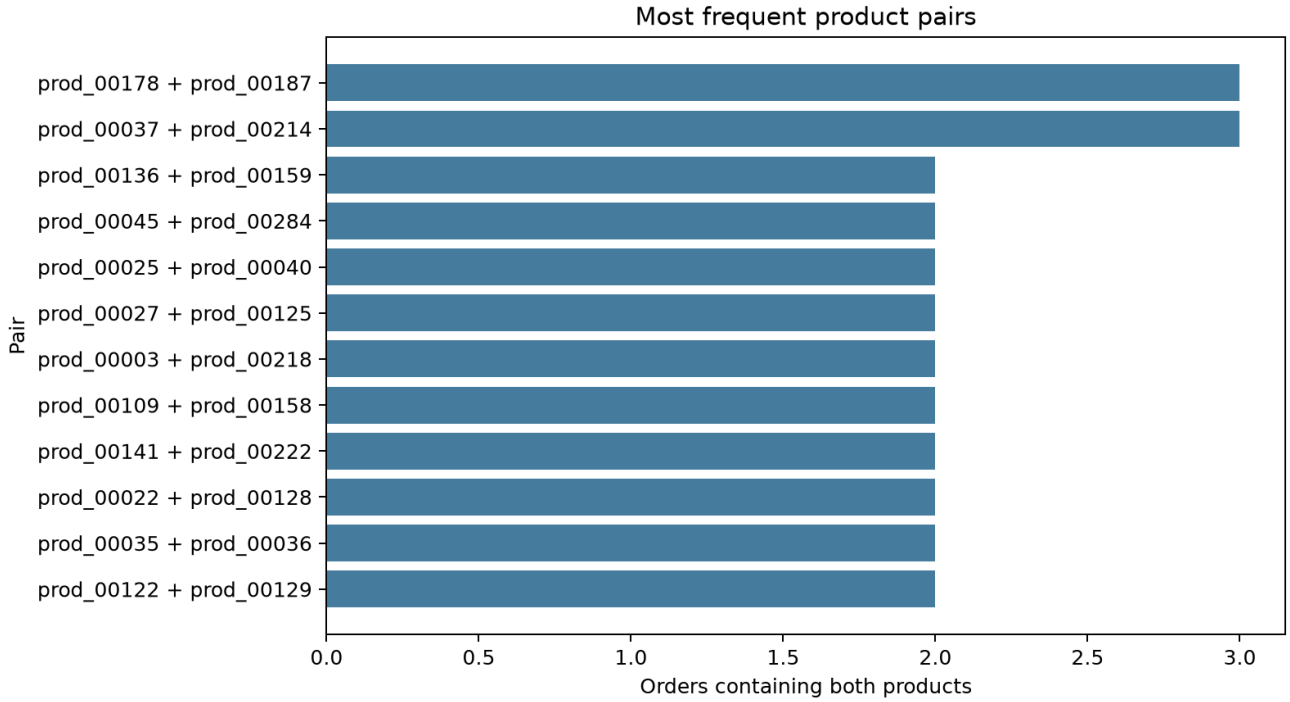


Figure 3. Conversion by referrer

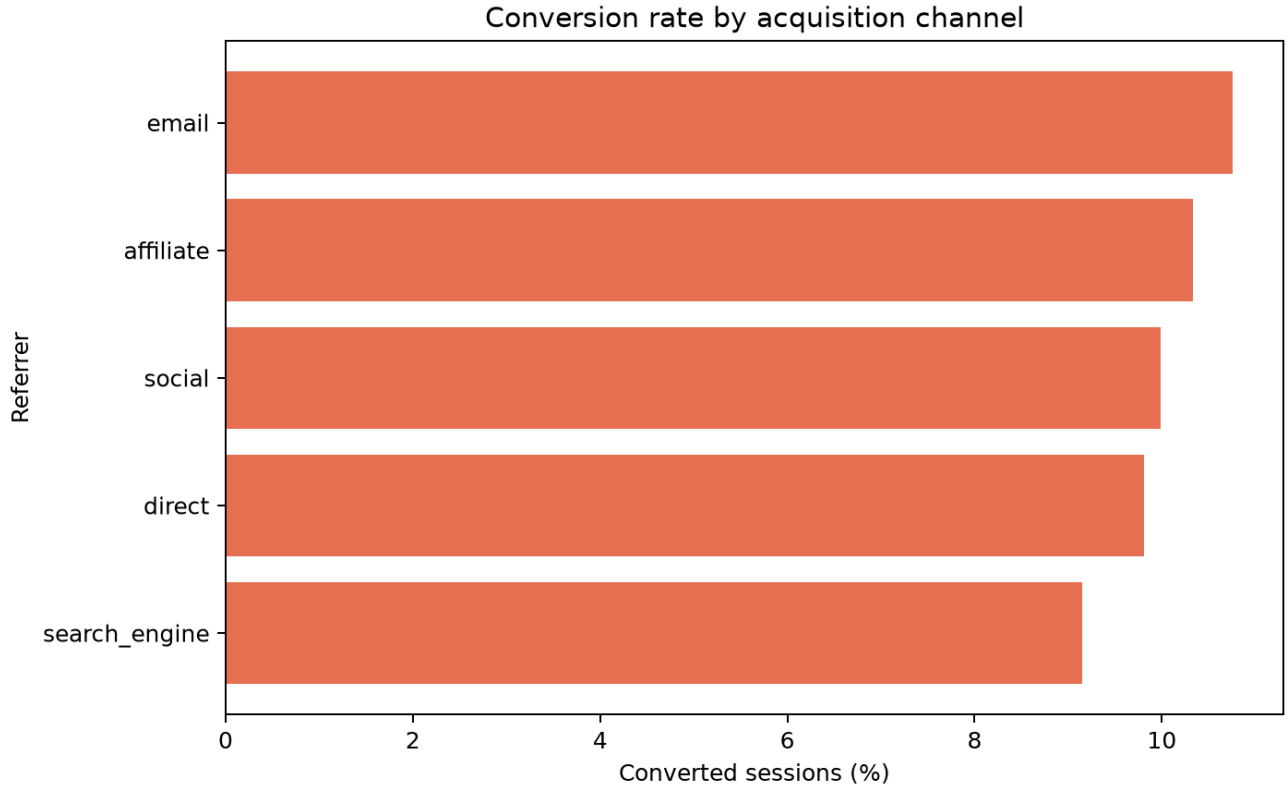
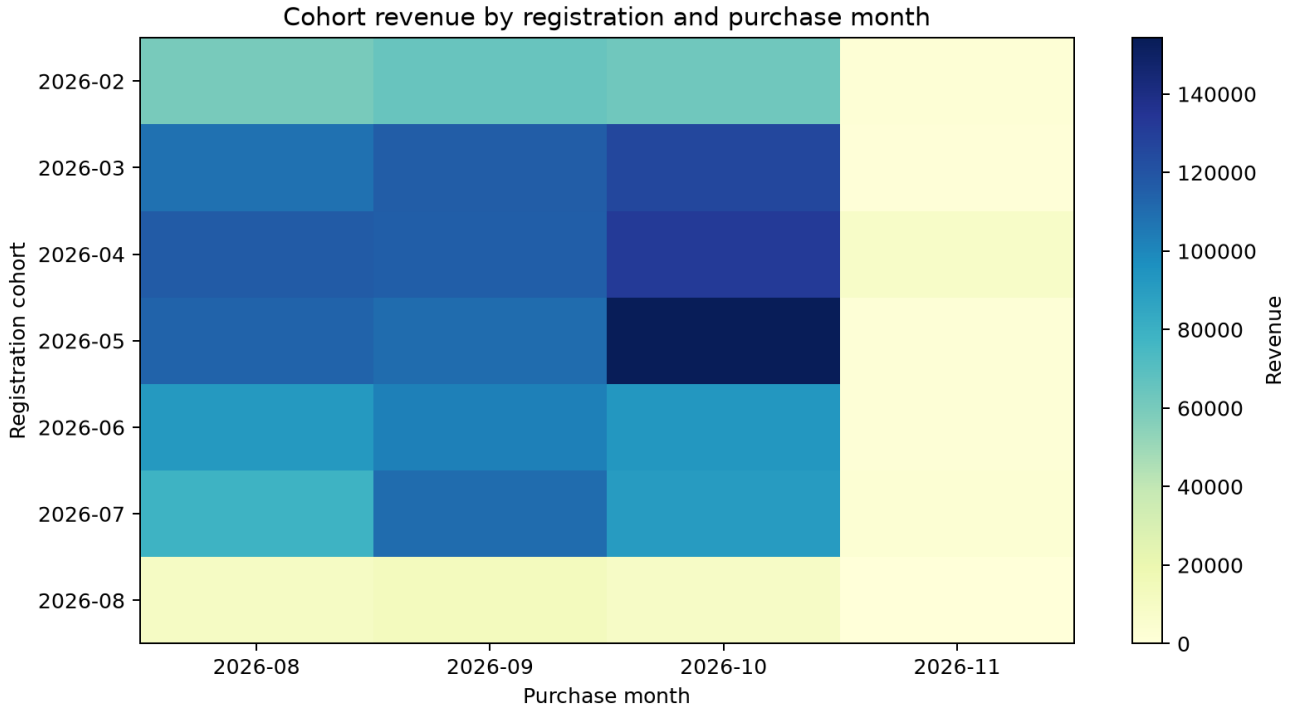


Figure 4. Revenue by registration and purchase cohort



The local sample contained 500 users, 300 products, 2,000 transactions, and 10,000 sessions. Spark produced 25 category rows, 2,682 product pairs, 27 cohort-month combinations, and 5 referrer rows. Generated data is excluded from version control.

6. Limitations and future work

The source is synthetic, so it lacks realistic marketing cost, product text, and customer demographics needed to measure profitability or causal lift. Product subcategory is inconsistently documented and is nullable in this implementation. The present recommendation score is co-occurrence, not personalized ranking; a next step is lift/confidence scoring or Spark ALS with explicit interaction data. Production HBase would use pre-splitting/salting if a few users became write hotspots, retention policies for raw events, and authenticated connectors.

7. Reproducibility

Commands and prerequisites are in `README.md`. The Spark run used Spark 3.5.9, Java 17, and the sample sizes listed above. MongoDB aggregation output and an HBase scan remain to be captured after those services are started; do not report those sections as executed until evidence is added.